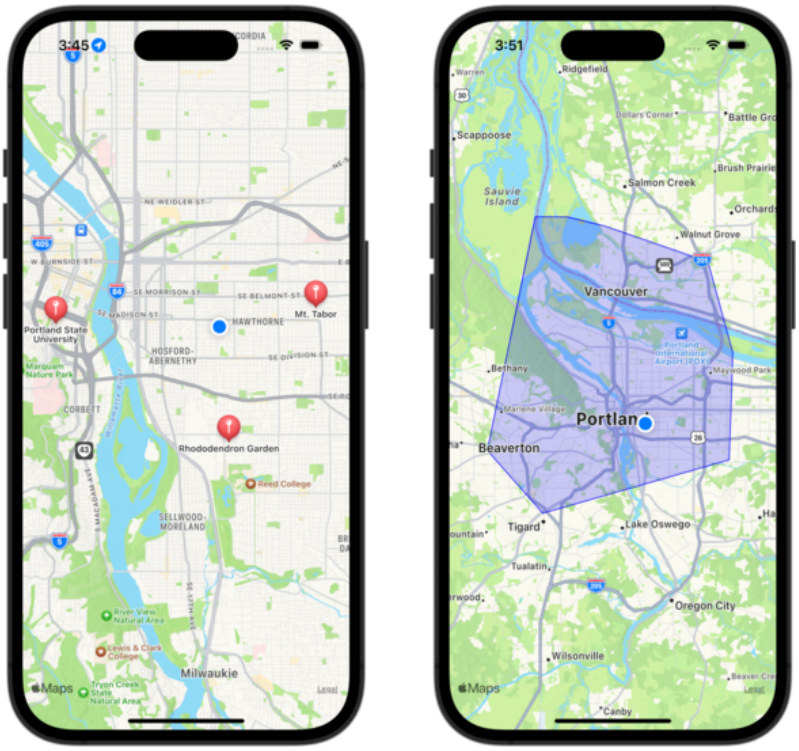


Finally, screens that interact with *native APIs* are often easier to build directly in Swift and Kotlin. I recently worked on an app that displayed HealthKit and Google Health Platform data. By keeping everything native, the data flowed directly from the API to Swift and Kotlin. Trying to render this via HTML would have required multiple round trips via JavaScript, overcomplicating the process.

When to Keep Using the Web View

On the other hand, there are times when I recommend to continue rendering content from Rails code.

Screens that change frequently, like settings or preferences, are easier to manage when rendered via HTML. Changes on the web are cheap relative to native ones. A SwiftUI or Jetpack Compose update often requires updates to the view *and* the API on the server. And each API change needs to ensure backward compatibility with all previous versions.